are particularly useful because they permit operations on the database to be made available to external applications, without exposing any of the internal details of the database.

Chapter 9 covers procedural extensions of SQL as well as many other new features of SQL:1999.

4.15 Summary

- Commercial database systems do not use the terse, formal query languages covered in Chapter 3. The widely used SQL language, which we studied in this chapter, is based on the formal relational algebra, but includes much "syntactic sugar."
- SQL includes a variety of language constructs for queries on the database. All the relational-algebra operations, including the extended relational-algebra operations, can be expressed by SQL. SQL also allows ordering of query results by sorting on specified attributes.
- View relations can be defined as relations containing the result of queries. Views are useful for hiding unneeded information, and for collecting together information from more than one relation into a single view.
- Temporary views defined by using the **with** clause are also useful for breaking up complex queries into smaller and easier-to-understand parts.
- SQL provides constructs for updating, inserting, and deleting information. A transaction consists of a sequence of operations, which must appear to be atomic. That is, all the operations are carried out successfully, or none is carried out. In practice, if a transaction cannot complete successfully, any partial actions it carried out are undone.
- Modifications to the database may lead to the generation of null values in tuples. We discussed how nulls can be introduced, and how the SQL query language handles queries on relations containing null values.
- The SQL data definition language is used to create relations with specified schemas. The SQL DDL supports a number of types including **date** and **time** types. Further details on the SQL DDL, in particular its support for integrity constraints, appear in Chapter 6.
- SQL queries can be invoked from host languages, via embedded and dynamic SQL. The ODBC and JDBC standards define application program interfaces to access SQL databases from C and Java language programs. Increasingly, programmers use these APIs to access databases.
- We also saw a brief overview of some advanced features of SQL, such as procedural extensions, catalogs, schemas and stored procedures.

Review Terms

- DDL: data definition language
- DML: data manipulation language
- **select** clause
- **from** clause
- **where** clause
- **as** clause
- Tuple variable
- **order** by clause
- Duplicates
- Set operations
  - **union, intersect, except**
- Aggregate functions
  - **avg, min, max, sum, count**
  - **group by**
- Null values
  - Truth value "unknown"
- Nested subqueries
- Set operations
  - $\{<, <=, >, >=\}$ { **some, all** }
  - **exists**
  - **unique**
- Views
- Derived relations (in **from** clause)
- **with** clause
- Database modification
  - **delete, insert, update**
  - View update
- Join types
  - Inner and outer join
  - left, right and full outer join
  - natural, using, and on
- Transaction
- Atomicity
- Index
- Schema
- Domains
- Embedded SQL
- Dynamic SQL
- ODBC
- JDBC
- Catalog
- Stored procedures

Exercises

4.1 Consider the insurance database of Figure 4.12, where the primary keys are underlined. Construct the following SQL queries for this relational database.

a. Find the total number of people who owned cars that were involved in accidents in 1989.

b. Find the number of accidents in which the cars belonging to "John Smith" were involved.

c. Add a new accident to the database; assume any values for required attributes.

d. Delete the Mazda belonging to "John Smith".

e. Update the damage amount for the car with license number "AABB2000" in the accident with report number "AR2197" to $3000.

4.2 Consider the employee database of Figure 4.13, where the primary keys are underlined. Give an expression in SQL for each of the following queries.

a. Find the names of all employees who work for First Bank Corporation.

person (*driver-id#*, *name*, *address*)
car (*license*, *model*, *year*)
accident (*report-number*, *date*, *location*)
owns (*driver-id#*, *license*)
participated (*driver-id*, *car*, *report-number*, *damage-amount*)

Figure 4.12 Insurance database.

employee (*employee-name*, *street*, *city*)
works (*employee-name*, *company-name*, *salary*)
company (*company-name*, *city*)
manages (*employee-name*, *manager-name*)

Figure 4.13 Employee database.

b. Find the names and cities of residence of all employees who work for First Bank Corporation.
c. Find the names, street addresses, and cities of residence of all employees who work for First Bank Corporation and earn more than $10,000.
d. Find all employees in the database who live in the same cities as the companies for which they work.
e. Find all employees in the database who live in the same cities and on the same streets as do their managers.
f. Find all employees in the database who do not work for First Bank Corporation.
g. Find all employees in the database who earn more than each employee of Small Bank Corporation.
h. Assume that the companies may be located in several cities. Find all companies located in every city in which Small Bank Corporation is located.
i. Find all employees who earn more than the average salary of all employees of their company.
j. Find the company that has the most employees.
k. Find the company that has the smallest payroll.
l. Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

4.3 Consider the relational database of Figure 4.13. Give an expression in SQL for each of the following queries.

a. Modify the database so that Jones now lives in Newtown.
b. Give all employees of First Bank Corporation a 10 percent raise.
c. Give all managers of First Bank Corporation a 10 percent raise.
d. Give all managers of First Bank Corporation a 10 percent raise unless the salary becomes greater than $100,000; in such cases, give only a 3 percent raise.
e. Delete all tuples in the *works* relation for employees of Small Bank Corporation.

4.4 Let the following relation schemas be given:

$$R = (A, B, C)$$
$$S = (D, E, F)$$

Let relations $r(R)$ and $s(S)$ be given. Give an expression in SQL that is equivalent to each of the following queries.

a. $\Pi_A(r)$
b. $\sigma_{B=17}(r)$
c. $r \times s$
d. $\Pi_{A,F}(\sigma_{C=D}(r \times s))$

4.5 Let $R = (A, B, C)$, and let r_1 and r_2 both be relations on schema R. Give an expression in SQL that is equivalent to each of the following queries.

a. $r_1 \cup r_2$
b. $r_1 \cap r_2$
c. $r_1 - r_2$
d. $\Pi_{AB}(r_1) \bowtie \Pi_{BC}(r_2)$

4.6 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Write an expression in SQL for each of the queries below:

a. $\{< a > \mid \exists\, b\, (< a, b > \in r \wedge b = 17)\}$
b. $\{< a, b, c > \mid < a, b > \in r \wedge < a, c > \in s\}$
c. $\{< a > \mid \exists\, c\, (< a, c > \in s \wedge \exists\, b_1, b_2\, (< a, b_1 > \in r \wedge < c, b_2 > \in r \wedge b_1 > b_2))\}$

4.7 Show that, in SQL, $<>$ **all** is identical to **not in**.

4.8 Consider the relational database of Figure 4.13. Using SQL, define a view consisting of *manager-name* and the average salary of all employees who work for that manager. Explain why the database system should not allow updates to be expressed in terms of this view.

4.9 Consider the SQL query

select $p.a1$
from p, $r1$, $r2$
where $p.a1 = r1.a1$ **or** $p.a1 = r2.a1$

Under what conditions does the preceding query select values of $p.a1$ that are either in $r1$ or in $r2$? Examine carefully the cases where one of $r1$ or $r2$ may be empty.

4.10 Write an SQL query, without using a **with** clause, to find all branches where the total account deposit is less than the average total account deposit at all branches,

a. Using a nested query in the **from** clauser.

b. Using a nested query in a **having** clause.

4.11 Suppose that we have a relation *marks*(*student-id*, *score*) and we wish to assign grades to students based on the score as follows: grade F if $score < 40$, grade C if $40 \leq score < 60$, grade B if $60 \leq score < 80$, and grade A if $80 \leq score$. Write SQL queries to do the following:

a. Display the grade for each student, based on the *marks* relation.
b. Find the number of students with each grade.

4.12 SQL-92 provides an n-ary operation called **coalesce**, which is defined as follows: **coalesce**$(A_1, A_2, \ldots, A_n)$ returns the first nonnull A_i in the list $A_1, A_2, \ldots, A_n$, and returns null if all of $A_1, A_2, \ldots, A_n$ are null. Show how to express the **coalesce** operation using the **case** operation.

4.13 Let a and b be relations with the schemas A(*name, address, title*) and B(*name, address, salary*), respectively. Show how to express a **natural full outer join** b using the **full outer join** operation with an **on** condition and the **coalesce** operation. Make sure that the result relation does not contain two copies of the attributes *name* and *address*, and that the solution is correct even if some tuples in a and b have null values for attributes *name* or *address*.

4.14 Give an SQL schema definition for the employee database of Figure 4.13. Choose an appropriate domain for each attribute and an appropriate primary key for each relation schema.

4.15 Write **check** conditions for the schema you defined in Exercise 4.14 to ensure that:

a. Every employee works for a company located in the same city as the city in which the employee lives.
b. No employee earns a salary higher than that of his manager.

4.16 Describe the circumstances in which you would choose to use embedded SQL rather than SQL alone or only a general-purpose programming language.

Bibliographical Notes

The original version of SQL, called Sequel 2, is described by Chamberlin et al. [1976]. Sequel 2 was derived from the languages Square Boyce et al. [1975] and Chamberlin and Boyce [1974]. The American National Standard SQL-86 is described in ANSI [1986]. The IBM Systems Application Architecture definition of SQL is defined by IBM [1987]. The official standards for SQL-89 and SQL-92 are available as ANSI [1989] and ANSI [1992], respectively.

Textbook descriptions of the SQL-92 language include Date and Darwen [1997], Melton and Simon [1993], and Cannan and Otten [1993]. Melton and Eisenberg [2000] provides a guide to SQLJ, JDBC, and related technologies. More information on SQLJ and SQLJ software can be obtained from http://www.sqlj.org. Date and Darwen [1997] and Date [1993a] include a critique of SQL-92.

Eisenberg and Melton [1999] provide an overview of SQL:1999. The standard is published as a sequence of five ISO/IEC standards documents, with several more parts describing various extensions under development. Part 1 (SQL/Framework), gives an overview of the other parts. Part 2 (SQL/Foundation) outlines the basics of the language. Part 3 (SQL/CLI) describes the Call-Level Interface. Part 4 (SQL/PSM) describes Persistent Stored Modules, and Part 5 (SQL/Bindings) describes host language bindings. The standard is useful to database implementers but is very hard to read. If you need them, you can purchase them electronically from the Web site http://webstore.ansi.org.

Many database products support SQL features beyond those specified in the standards, and may not support some features of the standard. More information on these features may be found in the SQL user manuals of the respective products. http://java.sun.com/docs/books/tutorial is an excellent source for more (and up-to-date) information on JDBC, and on Java in general. References to books on Java (including JDBC) are also available at this URL. The ODBC API is described in Microsoft [1997] and Sanders [1998].

The processing of SQL queries, including algorithms and performance issues, is discussed in Chapters 13 and 14. Bibliographic references on these matters appear in that chapter.